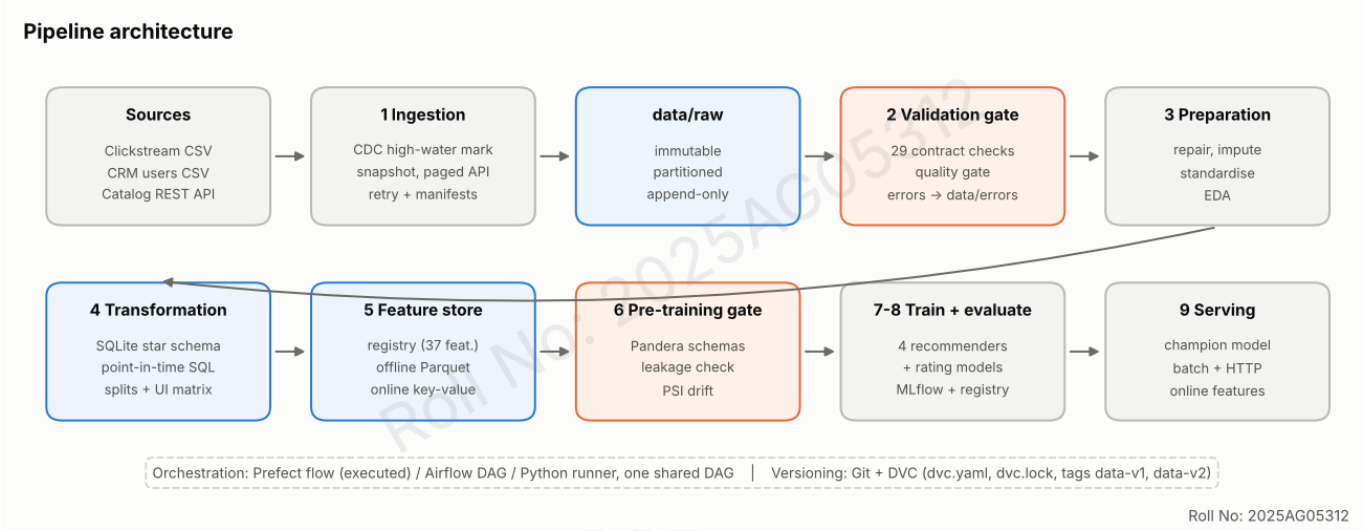
End-to-End Data Management Pipeline for a Recommendation System

Roll No: 2025AG05312 | AIML ZG529 Data Management for Machine Learning, Assignment 2 | Domain: e-commerce product recommendations

1. Problem and data

The pipeline recommends products to users of an online store from their past views, add-to-carts, purchases and ratings plus product metadata. Data is **clearly documented synthetic data** (`src/ingestion/generate_sources.py`, fixed seed): 1,500 users, 600 products in 12 categories and 77,217 raw events over 180 days, drawn from a latent-factor model (users prefer 1-3 categories, items have long-tail popularity) so collaborative filtering has real signal. Synthetic data was chosen over MovieLens so that the three source types the brief lists (CSV files, a database-style export and an API) and realistic defects could be controlled: duplicates, missing keys, orphan foreign keys, out-of-range ratings, malformed timestamps, casing drift, price-as-text type drift and negative prices are injected deliberately and must be caught downstream. After cleaning: 75,923 events, 40,220 user-item pairs, 95.5% sparsity, only 6.7% of events carry a rating, and the top 20% of items take 56% of interactions.

2. Architecture



Blue = storage layers, orange = blocking quality gates. Full diagram: docs/architecture.png.

3. Design decisions by stage

Ingestion. Three sources with three patterns: the clickstream log is pulled *incrementally* using an event_id high-water mark (CDC-style, so re-runs are idempotent); CRM users are a full snapshot (small dimension); the catalog is pulled from a paginated REST API with API-key auth and exponential backoff (the mock service returns a 503 on page 3 to prove the retry path). Raw data lands unmodified in an append-only, date-partitioned zone with a manifest per batch (row count, md5, schema) and an audit log. Raw is never cleaned in place, so any later stage can be replayed.

Validation (pre-ingestion gate). 29 row-level checks are declared once as data contracts in code, each tagged with a quality dimension (completeness, validity, uniqueness, consistency, referential integrity, timeliness) and a severity. *Critical* failures (null keys, unparseable or future timestamps, unknown event types, orphan user/product ids) are written to `data/errors/` with the failed check ids; *warnings* (bad ratings, casing, text prices) are kept and repaired in preparation. Dataset-level gate checks (schema, volume, key null rate, freshness, reject rate <= 5%) stop the pipeline if breached. Result: 915 duplicate events dropped, 379 events quarantined, data-quality score 99.80%, gate passed. Report: reports/validation/validation_report.html.

Preparation. Every repair is counted in reports/preparation_log.json: 35 invalid ages imputed (flagged), 6 text prices parsed, 15 bad prices replaced by the category median, 12 missing categories inferred from the title, 305 event types normalised and 53 invalid ratings nulled. Imputation flags are kept as columns so a model can tell real values from filled ones.

Transformation (ELT). Clean data is loaded into a SQLite star schema (dim_user, dim_product, dim_date, fact_interactions) whose CHECK and FOREIGN KEY constraints act as a last integrity line. Features are computed in SQL inside the warehouse, dbt-style, and every query is **point-in-time**: it only reads events before an *as_of* timestamp. Three snapshots are built: valid (cutoff - 28 d), train (cutoff - 14 d) and serve (all data). The time-based split files are train 61,383 events (to 2026-10-17) and test 14,540 events (2026-10-18 to 2026-10-31); valid is the last 14 days inside train and is used only for tuning. Derived datasets: user activity summary, item popularity, and a sparse 1500x600 user-item matrix where strength = sum of event weights (view 1, cart 3, purchase 5) with a 30-day half-life decay, plus (rating - 3).

Feature management. A custom feature layer mirrors Feast concepts: a registry of 37 features in 4 feature views (user behaviour, item popularity/quality, user-item interaction, user-category affinity) with type, description, owner, TTL and a definition hash that bumps the view version when its SQL changes; an offline Parquet store of every snapshot with a point-in-time join (pandas merge_asof) for training; and an online key-value store with the latest row per key for serving. Training and the prediction service read the same definitions, which removes training/serving skew from duplicated feature code. Catalog: reports/feature_catalog.md.

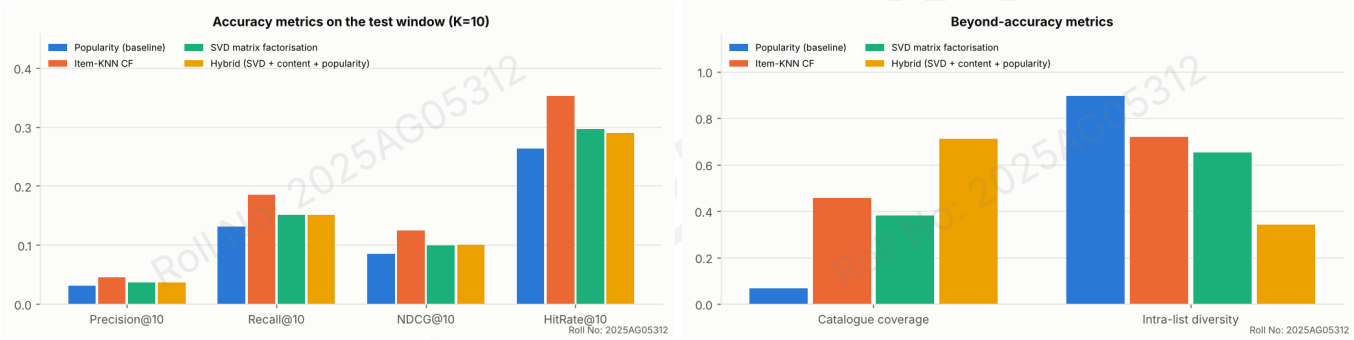
Pre-training gate. Pandera schemas validate the split files and feature snapshots; a leakage check asserts every train event precedes every test event and the training snapshot is not newer than the test window; PSI between train and test is 0.0001 for event-type mix and 0.0025 for category mix (warn above 0.10, fail above 0.25).

Versioning and orchestration. dvc.yaml declares all 10 stages with dependencies and outputs; dvc.lock pins md5 hashes, so `dvc repro` re-runs only what changed. Two data versions are tagged in Git: data-v1 (history to 17 Oct) and data-v2 (to 31 Oct, added by an incremental ingest of only the 14,798 new events); `git checkout data-v1 && dvc checkout` restores v1 exactly (reports/versioning/dvc_evidence.txt). One DAG definition drives three orchestrators: a Prefect 3 flow (executed, screenshot in docs/screenshots), an Airflow DAG, and a dependency-free Python runner. Gate tasks have zero retries, since retrying bad data does not fix it.

4. Models, evaluation protocol and results

Models: a **popularity baseline** (all-time weighted popularity blended with 7-day trend), **item-KNN collaborative filtering** (cosine item similarity, top-n neighbours), **truncated-SVD matrix factorisation** and a **hybrid** (SVD score + category-affinity content score + popularity prior). Hyper-parameters were tuned on the validation window only; each model family was then refit on the train snapshot and scored **once** on the test window, so the test set never influenced a choice. Relevance = an add-to-cart or purchase in the test window on an item the user had not touched before, evaluated over 921 users, K = 10. All runs are tracked in MLflow (params, metrics, feature-view versions and data md5s as lineage tags); the winner is retrained on all data, logged as a pyfunc model and registered as recsys_2025AG05312 with alias *champion*.

Model	Prec@10	Rec@10	NDCG@10	MAP@10	Hit@10	Coverage	Diversity	NDCG vs base
Popularity (baseline)	0.0312	0.1312	0.0851	0.0526	0.2628	0.068	0.896	+0.0%
Item-KNN CF (selected)	**0.0451**	**0.1850**	**0.1244**	**0.0789**	**0.3529**	**0.455**	**0.719**	**+46.1%**
SVD matrix factorisation	0.0367	0.1504	0.0990	0.0605	0.2964	0.382	0.653	+16.3%
Hybrid SVD + content	0.0359	0.1505	0.1007	0.0624	0.2899	0.710	0.341	+18.3%



Explicit ratings (supplementary, 958 test ratings): global-mean baseline RMSE 0.9688 / MAE 0.7923; regularised user+item bias model RMSE 0.9201 / MAE 0.7587. Full table and charts: reports/model/results_table.md, ranking_metrics.png, beyond_accuracy.png.

Reading the results. Item-KNN lifts NDCG@10 by 46% and Recall@10 from 0.131 to 0.185 over popularity, and recommends 46% of the catalogue against 7%. Absolute precision is low (about 0.45 hits per 10 recommendations) because only new items in a 14-day window count as hits; the lift over the baseline is the meaningful number. The beyond-accuracy metrics show trade-offs that accuracy alone hides: popularity has the highest intra-list diversity (0.90) only because the global best-sellers happen to span categories, while its coverage is tiny; the hybrid reaches the widest coverage (71%) but the lowest diversity (0.34) because the category-affinity term concentrates each list in the user's favourite categories. SVD degraded as factors grew (16 was best), a sign of over-fitting on a 96%-sparse matrix.

5. Limitations

(1) The data is synthetic and the generator is itself a latent-factor model, which favours collaborative filtering; results show the pipeline works, not how these models would rank on real traffic. Re-running on MovieLens or Amazon Reviews only needs a new ingestion adapter. (2) Single-node stack: SQLite, a dense item-item similarity matrix (O(items^2) memory) and in-memory pandas suit 600 items, not millions; production would use Postgres/BigQuery, Spark and approximate nearest neighbours. (3) The feature store is custom rather than Feast, and the online store is SQLite, not Redis. (4) The Airflow DAG is provided but only the Prefect flow and Python runner were executed. (5) Evaluation is offline only, on warm users; cold-start users fall back to popularity and were not scored, and there is no online A/B test. (6) No production monitoring of drift or model decay beyond the PSI gate; the next step would be scheduled drift checks on the serving snapshot that trigger retraining.

Tools. Python 3, pandas, NumPy, SciPy, scikit-learn, PyArrow/Parquet, SQLite, Pandera, MLflow (tracking + registry), Prefect 3, Apache Airflow (DAG file), Git + DVC, Matplotlib, ReportLab.